Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2026

IMPLEMENTACIÓN DE UNA API PARA LA AUTORIZACIÓN DE DOCUMENTOS TRIBUTARIOS ELECTRÓNICOS MEDIANTE PROGRAMACIÓN ORIENTADA A OBJETOS EN C#

202209463 – Carlos David Recinos Chip

Resumen

El presente trabajo describe el desarrollo de una solución informática orientada a la gestión y autorización de Documentos Tributarios Electrónicos (DTE) mediante una API implementada en C#. El sistema fue diseñado bajo los principios de la Programación Orientada a Objetos, permitiendo una estructura modular, escalable y mantenible. La aplicación procesa solicitudes enviadas en formato JSON, valida la información tributaria de emisores y receptores, verifica la integridad de los montos calculados y genera códigos únicos de autorización para los documentos aceptados.

Asimismo, el sistema incorpora la administración de un Registro Tributario Unificado (RTU), el almacenamiento persistente de la información y la generación de reportes estadísticos relacionados con la actividad tributaria procesada. Complementariamente, se desarrolló una interfaz web para la visualización de resultados y consultas de información. La implementación demuestra cómo la integración de tecnologías web modernas, servicios HTTP y bases conceptuales de ingeniería de software permite construir aplicaciones robustas para el procesamiento automatizado de información empresarial y tributaria.

Palabras clave

API, C#, JSON, DTE, Programación Orientada a Objetos

Abstract

This paper describes the development of a software solution focused on the authorization of Electronic Tax Documents (DTE) through an API implemented in C#. The system was designed following Object-Oriented Programming principles, providing a modular, scalable, and maintainable architecture. The application processes requests sent in JSON format, validates taxpayer information, verifies financial calculations, and generates unique authorization codes for approved documents.

Additionally, the system includes the administration of a Unified Tax Registry (RTU), persistent data storage, and the generation of statistical reports related to the processed tax activity. A web interface was also developed to display results and facilitate information queries. The implementation demonstrates how the integration of modern web technologies, HTTP services, and software engineering principles can be used to build robust applications for automated business and tax information processing.

Keywords

API, C#, JSON, DTE, Object-Oriented Programming

Introducción

La digitalización de los procesos tributarios ha permitido optimizar significativamente la gestión de documentos fiscales. Actualmente, los sistemas que administran Documentos Tributarios Electrónicos requieren mecanismos eficientes de validación, almacenamiento y consulta de información. En este contexto, las interfaces de programación de aplicaciones (API) constituyen una herramienta fundamental para facilitar la comunicación entre diferentes sistemas informáticos.

El presente ensayo analiza el desarrollo de una API para la autorización de DTE, implementada mediante programación orientada a objetos en lenguaje C#. Se abordan los principales componentes tecnológicos utilizados, así como los procesos de validación, almacenamiento y generación de reportes implementados. Asimismo, se evalúa la importancia de las arquitecturas basadas en servicios para resolver problemas reales relacionados con la administración tributaria y el intercambio de información digital.

Desarrollo del tema

1. Arquitectura general de la solución

La solución desarrollada se diseñó bajo una arquitectura cliente-servidor compuesta por dos componentes principales: una aplicación web desarrollada mediante Razor Pages y una API REST implementada utilizando ASP.NET Core. Esta organización permite separar la capa de presentación de la lógica de negocio, facilitando el mantenimiento y la escalabilidad del sistema. El proyecto solicitado para la simulación de autorización de Documentos Tributarios Electrónicos propone precisamente la comunicación entre un frontend y un backend mediante HTTP.

El frontend actúa como intermediario entre el usuario y la API, permitiendo cargar archivos JSON, realizar consultas y visualizar reportes. Por otra parte, el backend es responsable de ejecutar las validaciones, procesar la información recibida, generar autorizaciones y almacenar los datos necesarios para futuras consultas.

Esta arquitectura ofrece ventajas importantes, tales como la independencia entre componentes, la reutilización de servicios y la posibilidad de integrar clientes adicionales en el futuro sin modificar la lógica central del sistema

2. Gestión del Registro Tributario Unificado (RTU)

Uno de los componentes fundamentales implementados en el proyecto corresponde a la administración del Registro Tributario Unificado. El RTU almacena la información de los contribuyentes válidos que pueden participar en el proceso de emisión y recepción de facturas electrónicas. Según las especificaciones del proyecto, los contribuyentes son cargados mediante archivos JSON y posteriormente utilizados para validar las solicitudes de autorización.

La actualización del registro incluye tres escenarios principales. El primero ocurre cuando un contribuyente posee un NIT válido y no existe previamente en el sistema, lo que provoca su creación automática. El segundo escenario corresponde a la actualización de información de un contribuyente existente. Finalmente, si el NIT no cumple con el algoritmo de validación establecido, la información es descartada y no produce modificaciones sobre el registro.

La implementación de este módulo garantiza que únicamente contribuyentes válidos puedan participar en los procesos de autorización de documentos tributarios.

3. Algoritmo de validación de NIT

La validación de NIT representa una de las operaciones más importantes dentro del sistema debido a que permite verificar la autenticidad de los contribuyentes involucrados en cada transacción. El algoritmo utilizado calcula el dígito verificador mediante operaciones matemáticas sobre cada uno de

los caracteres que conforman el número de identificación tributaria.
Durante el procesamiento de una solicitud, tanto el NIT del emisor como el NIT del receptor son evaluados. Si alguno de ellos presenta inconsistencias o no cumple con la fórmula de verificación, la autorización es rechazada. Este procedimiento garantiza la confiabilidad de la información almacenada y evita el ingreso de datos incorrectos al sistema.
Además de validar la estructura matemática del NIT, la solución también verifica que el contribuyente exista previamente en el RTU. De esta manera se realiza una doble validación que incrementa la seguridad y precisión del proceso.

4. Procesamiento de solicitudes de autorización

El núcleo del sistema consiste en el procesamiento de solicitudes para la autorización de documentos tributarios electrónicos. Cada solicitud enviada contiene información relacionada con la fecha de emisión, referencia interna, NIT del emisor, NIT del receptor, valor de la factura, IVA y monto total.
Cuando la API recibe una solicitud, inicia una secuencia de validaciones encargadas de verificar la consistencia de cada dato. Primero se valida la existencia de los contribuyentes. Posteriormente se comprueba que la referencia no haya sido utilizada anteriormente. Finalmente se revisa que los cálculos financieros sean correctos.
Si todos los requisitos son cumplidos, el sistema procede a generar una autorización única. En caso contrario, la factura se registra como rechazada junto con el motivo correspondiente. Este comportamiento facilita la auditoría y el análisis posterior de errores.

5. Generación de códigos de autorización

La generación de autorizaciones constituye uno de los requisitos principales del proyecto. Cada documento aprobado recibe un código único compuesto por la fecha actual y un correlativo diario consecutivo. El formato permite identificar fácilmente la fecha de emisión de la autorización y garantiza que no se produzcan duplicados dentro de un mismo día.
Para lograr este comportamiento, el sistema mantiene un control interno del correlativo correspondiente a cada fecha. Cada vez que una autorización es aprobada, el contador incrementa su valor automáticamente. Este mecanismo asegura la unicidad de cada autorización emitida.
La automatización de este procedimiento permite procesar grandes cantidades de solicitudes de manera eficiente y confiable.

6. Persistencia y almacenamiento de información

La persistencia de datos es esencial para conservar la información procesada por la API. El proyecto requiere almacenar tanto los registros del RTU como los resultados de las autorizaciones realizadas.
El almacenamiento de la información permite realizar consultas históricas, generar reportes y mantener trazabilidad sobre cada documento procesado. Además, garantiza que la información permanezca disponible incluso después de reiniciar la aplicación.
La implementación de mecanismos de persistencia también facilita la construcción de estadísticas y reportes acumulativos, los cuales son utilizados posteriormente por los distintos módulos de consulta.

7. Generación de reportes tributarios

La API incorpora funcionalidades orientadas al análisis de la información procesada. Estas funcionalidades permiten obtener reportes por fechas específicas, rangos de tiempo y contribuyentes determinados.
Entre los reportes generados se encuentran los resúmenes de facturas autorizadas, el análisis de errores detectados durante el procesamiento y las estadísticas de emisores y receptores únicos. Asimismo, el sistema es capaz de calcular valores monetarios acumulados y presentar información relacionada con el IVA generado en cada período.
La disponibilidad de estos reportes transforma la información operacional en conocimiento útil para el seguimiento y control de la actividad tributaria.

8. Importancia de las APIs en los sistemas modernos

Las APIs representan uno de los elementos más importantes dentro del ecosistema tecnológico actual. Gracias a ellas, aplicaciones desarrolladas en diferentes plataformas pueden intercambiar información de manera eficiente utilizando estándares ampliamente aceptados.

En el contexto de este proyecto, la API opera como un intermediario encargado de recibir solicitudes, aplicar reglas de negocio y devolver respuestas estructuradas mediante JSON. Esta forma de trabajo facilita la integración con clientes web, aplicaciones móviles o sistemas empresariales.

La implementación desarrollada demuestra cómo el uso de APIs permite construir soluciones escalables, reutilizables y alineadas con las necesidades actuales del desarrollo de software empresarial.

9. Manejo de archivos JSON

El formato JSON se ha convertido en uno de los estándares más utilizados para el intercambio de información entre aplicaciones debido a su simplicidad, legibilidad y compatibilidad con múltiples tecnologías. En el proyecto desarrollado, este formato fue utilizado como medio principal para la comunicación entre el frontend y la API, permitiendo transmitir tanto la información del Registro Tributario Unificado como las solicitudes de autorización de facturas.

La utilización de JSON facilitó la serialización y deserialización de objetos dentro del sistema, permitiendo convertir estructuras de datos complejas en representaciones textuales fáciles de procesar. Gracias a ello fue posible manipular contribuyentes, facturas y respuestas de autorización de manera eficiente.

Además, el uso de mensajes JSON contribuye a la interoperabilidad del servicio, ya que cualquier aplicación capaz de generar este formato puede integrarse con la API desarrollada.

10. Aplicación de principios de ingeniería de software

Durante el desarrollo del proyecto se aplicaron diversos principios de ingeniería de software orientados a mejorar la calidad del producto final. La separación entre frontend y backend permitió distribuir responsabilidades claramente y mantener una arquitectura organizada. Asimismo, la implementación de clases específicas para cada entidad del sistema favoreció la mantenibilidad del código.

Otro aspecto relevante fue la modularización de las funcionalidades. Las operaciones relacionadas con validaciones, generación de autorizaciones, almacenamiento de información y generación de reportes fueron implementadas como componentes independientes. Esto reduce la complejidad del sistema y facilita futuras modificaciones.

La aplicación de estos principios permitió desarrollar una solución robusta capaz de adaptarse a nuevos requerimientos sin afectar significativamente la funcionalidad existente.

11. Seguridad y control de errores

Todo sistema que procesa información crítica debe contar con mecanismos que permitan detectar y manejar errores de forma adecuada. En este proyecto se implementaron diferentes validaciones para asegurar que únicamente se procesaran solicitudes que cumplieran con las reglas establecidas.

Entre los controles realizados se encuentran la validación de NIT, la verificación de contribuyentes registrados, el cálculo correcto del IVA, la comprobación de montos totales y la detección de referencias duplicadas. Estas verificaciones permiten evitar inconsistencias dentro de los registros almacenados y mejorar la confiabilidad de los resultados generados.

Además, la respuesta de la API proporciona información detallada acerca de las autorizaciones aprobadas y rechazadas, permitiendo identificar rápidamente posibles errores dentro de los datos enviados por los usuarios.

12. Aprendizajes y aportes del proyecto

El desarrollo de esta solución permitió aplicar de manera práctica diversos conocimientos adquiridos durante el curso de Introducción a la Programación y Computación 2. Entre ellos destacan la Programación Orientada a Objetos, el diseño de APIs REST, la manipulación de archivos JSON, el manejo de estructuras de datos y la construcción de aplicaciones web mediante tecnologías modernas.

Asimismo, el proyecto evidenció la importancia de realizar una adecuada planificación de la arquitectura antes de iniciar la implementación. La definición correcta de clases, relaciones y responsabilidades contribuyó significativamente a simplificar el desarrollo de funcionalidades complejas.

Finalmente, la experiencia obtenida demuestra que la combinación de conceptos teóricos y herramientas tecnológicas actuales permite construir sistemas capaces de resolver problemas reales relacionados con la gestión y procesamiento de información tributaria.

Conclusiones

1. El desarrollo de la API para la autorización de Documentos Tributarios Electrónicos permitió aplicar de forma práctica los principios de la Programación Orientada a Objetos mediante la creación de una arquitectura modular que facilita la organización del código, la reutilización de componentes y el mantenimiento de la aplicación.
2. La implementación de mecanismos de validación para NIT, referencias únicas, cálculos de IVA y montos totales demostró la importancia de establecer controles de integridad sobre los datos antes de almacenarlos o procesarlos, garantizando resultados confiables dentro del sistema.
3. El uso de servicios HTTP y mensajes JSON permitió construir una solución interoperable capaz de intercambiar información entre diferentes aplicaciones, evidenciando la relevancia de las APIs como herramienta fundamental en el desarrollo de software moderno.
4. La generación automática de códigos de autorización y la persistencia de la información contribuyeron a la automatización de procesos que normalmente requerirían una gestión manual, permitiendo un manejo más eficiente de los registros tributarios.
5. El proyecto fortaleció conocimientos relacionados con el desarrollo web, el diseño de arquitecturas cliente-servidor, la manipulación de archivos JSON y la implementación de soluciones basadas en .NET Core, demostrando la aplicación de conceptos académicos en un escenario similar a un entorno profesional.

Referencias bibliográficas

Microsoft. (2025). *ASP.NET Core documentation*. Microsoft Learn. https://learn.microsoft.com/aspnet/core

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures*. University of California.

Freeman, E., Freeman, E., Robson, E., & Sierra, K. (2021). *Head First Design Patterns* (2nd ed.). O'Reilly Media.

Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería del software: Un enfoque práctico* (9.ª ed.). McGraw-Hill.

Sommerville, I. (2019). *Software Engineering* (10th ed.). Pearson Education.

ANEXOS:

TABLA DE ACTIVIDADES

Semana	Actividad
Semana 1	**Análisis del enunciado, identificación de requerimientos, diseño de la arquitectura general del sistema, creación del repositorio y definición de las clases principales mediante Programación Orientada a Objetos.**
Semana 2	**Desarrollo de la API en ASP.NET Core, implementación del módulo de actualización del RTU, validación de NIT y procesamiento de archivos JSON.**
Semana 3	**Implementación de autorizaciones de DTE, generación de códigos únicos, almacenamiento de información, consultas de contribuyentes y generación de reportes solicitados.**
Semana 4	**Desarrollo del frontend con Razor Pages, integración con la API, elaboración de gráficas y reportes PDF, pruebas finales, corrección de errores y documentación del proyecto.**

DIAGRAMA DE CLASES:

